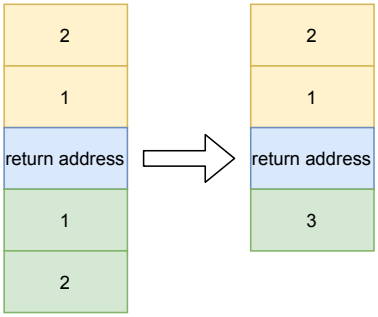
```
test: PEEK 1
      PEEK 3
      ADD
      RET
```

(a) Bytecode of `test`



(b) Stack at the execution of ADD

Figure 4: Compiling `test` to bytecode

PEEK *n***:** Reads the value n elements from the top (0 is top) and pushes it onto the stack. PEEK 0 is often called DUP in other interpreters, as it duplicates the value on top.

POP: Discards the value from the top of the stack.

CALL: Pops a value, which should be the address of a function, transferring control to it. The location after the call opcode is pushed onto the stack, marking the address where return should jump back.

JUMP *addr***:** Continues execution at *addr*.

JUMP_FALSE *addr***:** Pops a value from the stack. Jumps to *addr* if it is zero.

STOP: Stops the execution of the program.

ADD: Pops two values from the stack and pushes their sum.

3.3 Compiler

The IMP compiler consists of a lexer splitting the stream into a series of tokens, a parser constructing the Abstract Syntax Tree (AST) from the tokens, followed by the code generator mapping the AST to bytecode. Here the compiler is described briefly, more information is available among the sources of the project. To illustrate its operation, consider the following code fragment:

```
while (read_int()) {
  print_int(read_int() + read_int())
}
```

3.3.1 Lexer

The lexer traverses the sources character-by-character, splitting it into a series of tokens while also keeping track of the location of the tokens (line number, character number) in the source file. Implemented in `lexer.h` and `lexer.cpp`, it primarily exposes the `Next` method, which advances the stream and returns the next parsed token. The tokens themselves can either be standalone characters (such as parantheses or operators) or can carry additional information (as is the case with strings and named identifiers). Lexical analysis skips both whitespace and comments, as they carry no information relevant to building the AST. The prior example would be tokenised as follows:

```
WHILE, LPAREN, IDENT("read_int"), LPAREN, RPAREN, RPAREN, LBRACE,
IDENT("print_int"), LPAREN, IDENT("read_int"), LPAREN, RPAREN,
PLUS, IDENT("read_int"), LPAREN, RPAREN, RBRACE
```

3.3.2 Parser

The parser, implemented in `parser.h` and `parser.cpp` is a handwritten recursive-descent parser, inspecting one token at a time and constructing the appropriate AST nodes representing the input program. The parser relies on a single look-ahead token: the stream is advanced through calls to the `Next` method of the lexer, after which the kind of the returned token is inspected in order to identify the node that should be constructed, invoking the appropriate method validating and building it. Additionally, the parser is responsible for ensuring the syntax is valid, identifying errors and raising a descriptive `ParserError` to stop the compiler. Note that this compiler does not explicitly construct a parse tree: the stream of tokens is converted directly into AST nodes.

while (<cond>) <stmt>

On the prior example, the goal of the parser is to construct a node representing the `while` loop. Roughly, this node requires the presence of a keyword (`while`) and a nested expression between parenthesis specifying the loop condition. The loop body is a simple statement, optionally packed into a block. The fragment of code below shows the implementation of the method parsing while loops. The code first uses the `Check` method to ensure that the current token is indeed the `while` keyword, after which the stream is advanced asserting that the subsequent one is a parenthesis. The stream is advanced again to fetch a look-ahead token, recursively parsing an expression with the appropriate method. Since the expression parser moves the stream one token past the expression, on return the code ensures that it is followed by the appropriate closing parenthesis. Finally, the statement itself is parsed, building the node with the information gathered.

```
std::shared_ptr<WhileStmt> Parser::ParseWhileStmt()
{
  Check(Token::Kind::WHILE);
  Expect(Token::Kind::LPAREN);
  lexer_.Next();
  auto cond = ParseExpr();
  Check(Token::Kind::RPAREN);
  lexer_.Next();
  auto stmt = ParseStmt();
  return std::make_shared<WhileStmt>(cond, stmt);
}
```

3.3.3 Codegen

The code generator receives the AST from the parser and must generate a sequence of instructions specifying the behaviour of the program. In contrast with the tree structure of the AST, the opcodes are flattened and laid out sequentially in memory. Control flow is implemented using labels and conditional jumps: labels are created to identify specific points in the program, which are used as operands to instructions which transfer control to them. The methods lowering the nodes rely on a set of helpers (`Emit*`) to encode instructions.

```
void Codegen::LowerWhileStmt(const Scope &scope, const WhileStmt &whileStmt)
{
  auto entry = MakeLabel();
  auto exit = MakeLabel();

  EmitLabel(entry);
  LowerExpr(scope, whileStmt.GetCond());
  EmitJumpFalse(exit);
  LowerStmt(scope, whileStmt.GetStmt());
  EmitJump(entry);
  EmitLabel(exit);
}
```

The function above translates a while loop to bytecode. First, two labels are created: one to identify the entry point and one to point to the exit. The `EmitLabel` call pins the label to the address past the last emitted instruction, allowing other instructions to reference this location later on. The condition is then lowered, followed by a conditional jump which exits the loop when the check fails. At this point, the address of the exit label is unknown, as it has not yet been emitted: to account for forward references, the location of the operand is recorded as a fixup to be adjusted later on, once the label is known. Following the jump, the body is generated and closed with a backwards jump back to the entry and to the next iteration of the condition check. This is a backwards jump to a known label, allowing the correct address to be emitted straight away without the need for a fixup. The sequence of bytes encoding the instructions is packed into a `Program` defined in `program.h`, to be passed on to the interpreter for execution.